

AgroVivo 2.1 — Product Requirements Document

Digitalização, Automação FSM e Mapeamento GIS de Planos de Manejo Orgânico

Versão: 2.1.0 (Atualização Arquitetural Pós-Pivot)

Status: Ativo / Em Produção

1. Visão Geral e Estratégia

A certificação orgânica exige documentação complexa (18 secções do Plano de Manejo Orgânico). A solução AgroVivo evoluiu de um simples bot "Voice-to-JSON" para um **ecossistema cognitivo e geoespacial** de alta performance.

A Solução AgroVivo 2.1:

1. O produtor regista atividades diárias no WhatsApp por voz ou texto.
2. A FSM (Máquina de Estados) em **Golang** classifica a intenção via **Google Gemini API**.
3. Módulo RAG responde a dúvidas sobre a legislação (Lei 10.831) instantaneamente.
4. O Frontend em **React + MapLibre** renderiza a fazenda em WebGL, associando os registos aos polígonos reais dos talhões.
5. Validação ativa via "Output Judge" garante compliance antes de qualquer registo ou resposta.

2. Arquitetura do Sistema (Pivot Arquitetural)

A arquitetura transitou de Python/Flask para **Golang (Clean Architecture / Ports & Adapters)**, garantindo latência ultrabaixa e processamento assíncrono avançado.

Diagrama de Arquitetura

- **Canais de Entrada:** WhatsApp via Evolution API e WPPConnect (Adaptadores em Go), Web PWA (React).
- **Orquestração (Backend):** pmo-bot-go gerindo Workers assíncronos, paralelização via errgroup e FSM Cognitiva.
- **IA e NLP:** Google Gemini API (Classificador, Orchestrator, Output Judge) com fallback para OpenRouter. RAG via ledongthuc/pdf.
- **Frontend (GIS):** React 19 + MapLibre GL JS + WebGL para renderização de propriedades e talhões (setFeatureState para performance).
- **Persistência e Filas:** PostgreSQL (Supabase) + Redis via Docker Compose.

Stack Tecnológica Atualizada

Camada	Tecnologia	Justificativa do Pivot
Frontend / GIS	React 19 + MapLibre	Performance GPU/WebGL para renderização de polígonos complexos sem encravar a DOM.
Backend	Golang (pmo-bot-go)	Substituiu o Python para gerir <i>goroutines</i> , concorrência (errgroup) e baixa latência (menos de 4s).
Integração WhatsApp	Evolution API / WPPConnect	Abstração via "Ports and Adapters" para redundância e resiliência no envio de mensagens.
IA / Cognição	Google Gemini API	Maior velocidade no Time-to-First-Token, substituindo Llama/Groq para orquestração da FSM.
Base de Dados	Supabase (PostgreSQL)	Mantido. Uso extensivo de JSONB e suporte futuro para PostGIS.

3. Contrato de Dados (Injeção Geoespacial)

Além da estrutura JSONB referenciada na V2.0, o sistema agora integra dados geoespaciais e de telemetria.

Estrutura de Talhões (Módulo GIS)

Os talhões deixam de ser puramente textuais e passam a incorporar coordenadas (LngLat) e Bounding Boxes para o MapLibre.

```
{
  "talhao_id": "uuid",
  "nome": "Canteiro 3 - Hortaliças",
  "geometria": {
    "type": "Polygon",
    "coordinates": [[[lng, lat], [lng, lat], ...]]
  },
  "area_hectares": 1.2
}
```

}

Telemetria de Latência

O backend Go regista a latência de cada passo FSM no server.log (ex: [TRACING] Google Gemini API: 1.5s) e grava logs assíncronos no Supabase sem bloquear a thread do utilizador.

4. Requisitos Funcionais Evoluídos

4.1 Módulo FSM e Guardrails (Backend Go)

- **US-FSM-01 (Bypass Seletivo):** O sistema deve contornar o *Output Judge* para intenções do tipo CHAT, poupando ~830ms de latência.
- **US-FSM-02 (RAG em Tempo Real):** O sistema deve ler manuais em PDF e a Lei 10.831 e responder às perguntas do produtor através de uma intenção específica (RAG).
- **US-FSM-03 (Processamento Concorrente):** Validações de cota e extrações de intenção devem correr paralelamente usando errgroup.

4.2 Módulo GIS e Caderno de Campo (Frontend)

- **US-GIS-01 (Renderização Segura de Mapas):** O MapController deve usar blocos try/catch e validações anti-NaN (safeFitBounds) para evitar o bloqueio (*White Screen of Death*) na montagem de mapas sem coordenadas.
- **US-GIS-02 (Estado GPU-First):** A pintura de talhões selecionados deve usar map.setFeatureState em vez de re-renderizações do React (useState) para lidar com +40.000 ficheiros e grandes polígonos de forma fluida.

5. Requisitos Não-Funcionais e Performance

Métrica	Target V2.1 (Go)	Observações
Latência Total da FSM (processAIJob)	< 4s	Conseguido através de Bypass do Output Judge e Paralelização.
Latência Setup (Go Internals)	< 1s	Tempo de CPU/Go, excluindo chamadas de rede da IA.
Renderização do Mapa (Frontend)	60 FPS	Garantido via WebGL e offloading do estado do React.

6. Roadmap de Expansão (Atualizado)

Fase 2.1 (Em Produção / Concluído)

- Migração completa para Backend Golang (Ports & Adapters).
- Implementação da FSM Cognitiva e Integração Gemini API.
- Módulo RAG para Legislação Orgânica ativo.
- Renderização GIS via MapLibre no Frontend.

Fase 2.2 (Próximos Passos)

Feature	Descrição	Prioridade
Otimização do Time-To-First-Token	Explorar modelos mais leves (ex: Gemini Flash-Lite) para o Router/Classificador FSM.	Alta
Sincronização Offline do GIS	Cache de <i>tiles</i> do MapLibre e polígonos no IndexedDB para uso em áreas sem cobertura.	Média
Exportação de Mapas no PDF	Incluir um print estático da propriedade no PMO PDF gerado para a certificadora.	Baixa